

HERANÇA MÚLTIPLA

Prof. André Backes

Definição

- Permite criar uma classe derivada que combina funcionalidades de diferentes hierarquias de classes
 - Permite que uma classe herde características (atributos e métodos) de mais de uma classe base ao mesmo tempo

Declaração

- Parecida com a da herança simples
- Basta indicar as classes bases e seus modos de acesso separados por vírgula
 - Os modos de acesso podem ser **public**, **protected** ou **private**
 - Definem como os membros da base serão herdados

Declaração

Herança simples

```
class Derivada : public Base {  
    // Métodos e atributos da classe  
};
```

Herança múltipla

```
class Derivada : public Base1, public Base2 {  
    // Métodos e atributos da classe  
};
```

Criando uma herança múltipla

- Imagine que queremos criar uma classe que represente um professor
- Considere também que o nosso sistema já possui algumas classes implementadas
 - Pessoa
 - Empregado

Criando uma herança múltipla

- Pessoa e Empregado são bases independentes
 - Cada uma encapsula um aspecto do domínio do problema
 - Pessoa: dados de identidade (nome e idade)
 - Empregado: vínculo trabalhista (salario)

```
class Pessoa {
protected:
    string nome;
    int idade;
public:
    Pessoa(string n, int i) : nome{n}, idade{i} {}
    void mostrarPessoa() {
        cout << "Nome: " << nome << ", Idade: " << idade << "\n";
    }
};

class Empregado {
protected:
    double salario;
public:
    Empregado(double s) : salario{s} {}
    void mostrarEmpregado() {
        cout << "Salário: " << salario << "\n";
    }
};
```

Criando uma herança múltipla

- Um professor “é uma” Pessoa e também “é um” Empregado
 - Faz sentido que herde as características de Pessoa e Empregado

```
class Professor : public Pessoa, public Empregado {
private:
    string area;
public:
    Professor(string n, int i, double s, string a)
        : Pessoa(n, i), Empregado(s), area{a} {}

    void mostrarProfessor() {
        // Método da classe Pessoa
        mostrarPessoa();
        // Método da classe Empregado
        mostrarEmpregado();
        cout << "Área de atuação: " << area << "\n";
    }
};

int main() {
    Professor p("Ana Silva", 40, 3500.00, "Matemática");
    p.mostrarProfessor();

    return 0;
}
```

Criando uma herança múltipla

- Vários conceitos de herança vistos anteriormente estão presentes
 - Visibilidade de atributos
 - Delegação de construtores
 - Reuso de funcionalidades para compor uma nova
 - mostrarPessoa() e mostrarEmpregado()

```
class Professor : public Pessoa, public Empregado {
private:
    string area;
public:
    Professor(string n, int i, double s, string a)
        : Pessoa(n, i), Empregado(s), area{a} {}

    void mostrarProfessor() {
        // Método da classe Pessoa
        mostrarPessoa();
        // Método da classe Empregado
        mostrarEmpregado();
        cout << "Área de atuação: " << area << "\n";
    }
};

int main() {
    Professor p("Ana Silva", 40, 3500.00, "Matemática");
    p.mostrarProfessor();

    return 0;
}
```


Acesso e superposição de métodos

- A superposição ocorre quando um método na classe derivada tem o mesmo nome de um método na classe base
 - Permite mudar a implementação da função da classe base na derivada
 - A superposição oculta todas as versões do método na classe Base
 - Mesmo que tenham uma lista de parâmetros ou tipo de retorno diferentes

Acesso e superposição de métodos

- A diretiva **using** permite acessar as funções sobrecarregadas ocultas de uma classe base
 - **imprimir()**: implementação da derivada
 - **imprimir(5)**: implementação da classe Base1
 - Agora está visível novamente

```
class Base1 {
public:
    void imprimir() {
        cout << "Imprimir() da Base1\n";
    }
    void imprimir(int n) {
        cout << "Imprimir() da Base1: " << n << "\n";
    }
};

class Base2 {
public:
    void imprimir() {
        cout << "Imprimir() da Base2\n";
    }
};

class Derivada : public Base1, public Base2 {
public:
    using Base1::imprimir;

    void imprimir() {
        cout << "Imprimir() da Derivada\n";
    }
};

int main() {
    Derivada d;
    d.imprimir();
    d.imprimir(5);

    return 0;
}
```

Acesso e superposição de métodos

- Outro problema: ambiguidade de nomes
 - Classes base possuem membros com o mesmo nome
 - Se Derivada não implementa esse método, todas as versões das classes bases são visíveis
 - Qual chamar?

```
class Base1 {
public:
    void imprimir() {
        cout << "Imprimir() da Base1\n";
    }
    void imprimir(int n) {
        cout << "Imprimir() da Base1: " << n << "\n";
    }
};

class Base2 {
public:
    void imprimir() {
        cout << "Imprimir() da Base2\n";
    }
};

class Derivada : public Base1, public Base2 {
public:
    void mostrar() {
        cout << "Imprimir() da Derivada\n";
    }
};

int main() {
    Derivada d;
    //Erro de compilação: ambiguidade de nome
    //d.imprimir();

    return 0;
}
```

Acesso e superposição de métodos

- Solução para a ambiguidade de nomes
 - Usar o operador de resolução de escopo ::
 - Indica explicitamente de qual classe base o método deve ser usado
 - **Base1::imprimir()**
 - **Base2::imprimir()**

```
class Base1 {
public:
    void imprimir() {
        cout << "Imprimir() da Base1\n";
    }
    void imprimir(int n) {
        cout << "Imprimir() da Base1: " << n << "\n";
    }
};

class Base2 {
public:
    void imprimir() {
        cout << "Imprimir() da Base2\n";
    }
};

class Derivada : public Base1, public Base2 {
public:
    void mostrar() {
        cout << "Imprimir() da Derivada\n";
    }
};

int main() {
    Derivada d;
    //Erro de compilação: ambiguidade de nome
    //d.imprimir();

    d.Base1::imprimir();
    d.Base2::imprimir();

    return 0;
}
```

Ordem do construtor e destrutor

- Processo de construção é feito de cima para baixo na hierarquia
 - Classe Base, depois Derivada
 - Mas temos duas bases... Qual vem primeiro?

```
class Base1 {
public:
    Base1() {
        cout << "Construtor da Base 1\n";
    }
};

class Base2 {
public:
    Base2() {
        cout << "Construtor da Base 2\n";
    }
};

class Derivada : public Base1, public Base2 {
public:
    Derivada() {
        cout << "Construtor da Derivada\n";
    }
};

int main() {
    Derivada de;

    return 0;
}
```

Ordem do construtor e destrutor

- Ordem de chamada dos construtores das bases

- É determinada pela ordem em que as bases aparecem na lista de herança
 - Primeiro Base1, depois Base2
- Essa ordem não depende da lista de inicialização no construtor de Derivada
 - Se existir

- Saída

Construtor de Base1

Construtor de Base2

Construtor de Derivada

```
class Base1 {
public:
    Base1() {
        cout << "Construtor da Base 1\n";
    }
};

class Base2 {
public:
    Base2() {
        cout << "Construtor da Base 2\n";
    }
};

class Derivada : public Base1, public Base2 {
public:
    Derivada() {
        cout << "Construtor da Derivada\n";
    }
};

int main() {
    Derivada de;

    return 0;
}
```

Ordem do construtor e destrutor

- A destruição ocorre no sentido inverso da criação

- Derivada, depois Bases
- Ordem estritamente inversa à construção
 - Derivada, Base2, Base1
- Saída

Construtor de Base1

Construtor de Base2

Construtor de Derivada

Destrutor de Derivada

Destrutor de Base2

Destrutor de Base1

```
class Base1 {
public:
    Base1() {
        cout << "Construtor da Base 1\n";
    }
    ~Base1() {
        cout << "Destrutor da Base 1\n";
    }
};

class Base2 {
public:
    Base2() {
        cout << "Construtor da Base 2\n";
    }
    ~Base2() {
        cout << "Destrutor da Base 2\n";
    }
};

class Derivada : public Base1, public Base2 {
public:
    Derivada() {
        cout << "Construtor da Derivada\n";
    }
    ~Derivada() {
        cout << "Destrutor da Derivada\n";
    }
};

int main() {
    Derivada de;

    return 0;
}
```

Herança múltipla

Vantagens

- Reuso de código
 - Permite combinar funcionalidades já implementadas em diferentes classes
- Modelagem mais flexível
 - Útil quando uma classe pertence a mais de uma hierarquia
 - ImpressoraScanner poderia herdar de Impressora e de Scanner
- Composição de comportamentos sem criar classes monolíticas
 - Com cada classe base especializada em um aspecto específico

Desvantagens

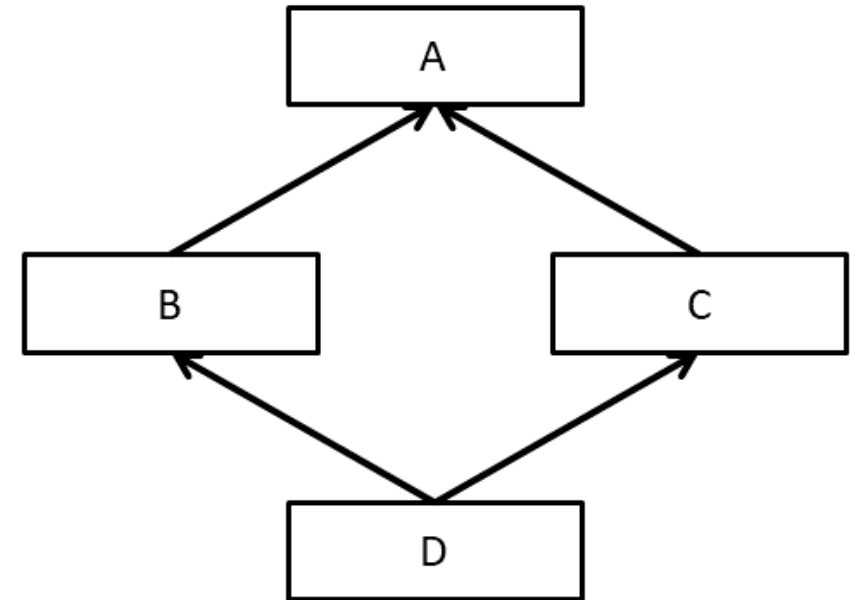
- Ambiguidade de nomes
 - Duas classes base possuem métodos ou atributos com o mesmo identificador
- Problema do diamante (*diamond problem*)
 - Pode gerar múltiplas cópias da classe ancestral, causando redundância e inconsistência nos dados
- Maior complexidade
 - Conforme a hierarquia cresce, o código se torna mais difícil de compreender e manter

O problema do diamante

- Ocorre quando duas classes base herdam de uma mesma classe ancestral, e a classe derivada final herda de ambas. Isso causa duplicação dos membros da classe ancestral
 - Esse nome vem do formato do diagrama de herança resultante
 - Um dos desafios clássicos da herança múltipla em C++

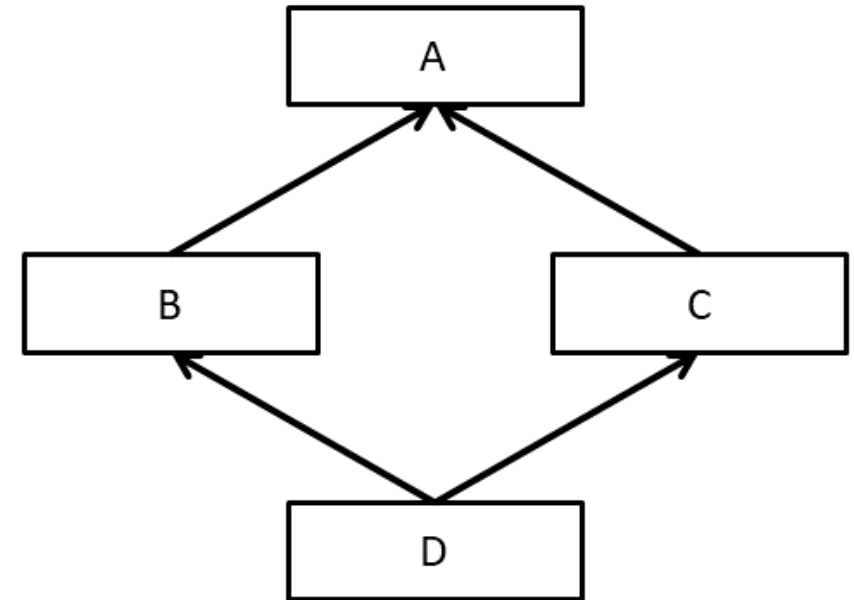
O problema do diamante

- Estrutura
 - As classes B e C herdam da classe A
 - A classe D herda de B e C
- Problema
 - Se a classe D usa um método da classe A, então qual versão do método ela herda: a versão da classe B ou a da classe C?



O problema do diamante

- Essa duplicação causar diversos problemas:
 - Ambiguidade
 - Devo acessar o membro de A herdado por B ou de C?
 - Redundância de dados
 - Se A tiver atributos, eles serão duplicados em D
 - Maior gasto com memória
 - Inconsistência lógica
 - Temos duas cópias de A, uma pode ser alterada enquanto a outra não
 - Isso resulta em comportamento inesperado



O problema do diamante | Exemplo

- Ao instanciar um objeto da classe D
 - O compilador chamar o construtor da classe A duas vezes
 - Na criação do subobjeto B
 - Na criação do subobjeto C
 - B e C tem o seu próprio **valor**
 - Quando o objeto D tenta acessar, o compilador sinaliza ambiguidade
 - Não há como escolher automaticamente qual versão do atributo usar

```
class A {  
public:  
    int valor;  
    A() {  
        cout << "Construtor de A\n";  
    }  
};  
  
class B : public A {};  
class C : public A {};  
class D : public B, public C {};  
  
int main() {  
    D obj;  
    // ERRO: ambiguidade  
    // Existem duas cópias de 'valor'  
    // obj.valor = 10;  
  
    return 0;  
}
```

O problema do diamante | Exemplo

- A ambiguidade não impede o acesso ao atributo
 - Ela apenas exige que seja explícito sobre qual caminho de herança percorrer
 - Isso é feito usando o operador de resolução de escopo
 - B::valor
 - C::valor

```
class A {
public:
    int valor;
    A() {
        cout << "Construtor de A\n";
    }
};

class B : public A {};
class C : public A {};
class D : public B, public C {};

int main() {
    D obj;
    // ERRO: ambiguidade
    // Existem duas cópias de 'valor'
    // obj.valor = 10;

    // Acesso explícito
    // Redundância de dados
    obj.B::valor = 10;
    obj.C::valor = 10;

    return 0;
}
```

Herança Virtual

- A solução oferecida pelo C++ para resolver o problema do diamante é usar herança virtual
 - Quando uma classe herda de outra virtualmente, o compilador garante que apenas uma cópia única da classe base seja herdada, mesmo que a herança passe por múltiplos caminhos

Herança Virtual

- O compilador trata as classes base virtuais de maneira especial
 - Em vez de incluir diretamente os membros da classe base em cada ramo da herança, o compilador mantém apenas uma referência comum para a base virtual.
 - Todas as classes derivadas de uma hierarquia em diamante compartilham a mesma instância da classe base virtual

Herança Virtual | Declaração

Herança

```
class Derivada : public Base {  
    // Métodos e atributos da classe  
};
```

Herança Virtual

```
class Derivada : virtual public Base {  
    // Métodos e atributos da classe  
};
```


Herança Virtual | Exemplo

- Estrutura
 - As classes B e C herdam da classe A, mas herdam virtualmente
 - **virtual public A**
 - A classe D herda de B e C
- Ao instanciar um objeto da classe D
 - O construtor da classe A é chamado uma única vez
 - Atributo **valor** agora é único
 - Não duplicação de atributo e ambiguidades

```
class A {  
public:  
    int valor;  
    A() {  
        cout << "Construtor de A\n";  
    }  
};  
  
class B : virtual public A {};  
class C : virtual public A {};  
class D : public B, public C {};  
  
int main() {  
    D obj;  
    // Atributo único  
    obj.valor = 10;  
  
    return 0;  
}
```

Herança Virtual | Inicialização

- Afeta a ordem de construção e a inicialização da classe base compartilhada
 - A classe mais derivada se torna responsável por construir a classe base virtual
 - A classe base A possui um construtor parametrizado
 - A classe D, classe mais derivada, deve inicializar a classe A em sua lista de inicialização
 - Ela que realmente cria o objeto

```
class A {
public:
    int valor;
    A(int v): valor{v}{
        cout << "Construtor de A\n";
    }
};

class B : virtual public A {
public:
    B(int v): A(v){
        cout << "Construtor de B\n";
    }
};

class C : virtual public A {
public:
    C(int v): A(v){
        cout << "Construtor de C\n";
    }
};

class D : public B, public C {
public:
    D(int v): A(v), B(v), C(v) {
        cout << "Construtor de D\n";
    }
};

int main() {
    D obj(20);
    cout << obj.valor;

    return 0;
}
```

Herança Virtual | Inicialização

- As tentativas das classes B e C de inicializar A(v) em seus próprios inicializadores são ignoradas
 - A inicialização da classe A pelas classes B e C somente ocorre se elas forem as mais derivadas
 - Ao fazer a instanciação de um objeto da classe B
 - B b(10);
 - B passa a ser a classe “mais derivada” e então a sua inicialização de A, A(v), é usada

```
class A {
public:
    int valor;
    A(int v): valor(v){
        cout << "Construtor de A\n";
    }
};

class B : virtual public A {
public:
    B(int v): A(v){
        cout << "Construtor de B\n";
    }
};

class C : virtual public A {
public:
    C(int v): A(v){
        cout << "Construtor de C\n";
    }
};

class D : public B, public C {
public:
    D(int v): A(v), B(v), C(v){
        cout << "Construtor de D\n";
    }
};

int main() {
    D obj(20);
    cout << obj.valor;

    return 0;
}
```

Herança Virtual | Cuidados

- Permite a modelagem de hierarquias complexas
 - Evitando o problema do diamante e a duplicação de códigos e possíveis ambiguidade
- Porém aumenta o custo e a complexidade do programa
 - Ajustes de ponteiro, tabelas internas para localizar a base virtual, e possivelmente mais espaço por objeto
 - Também torna a semântica de inicialização mais complexa
 - A classe mais derivada passa a ser responsável por construir a base virtual

Material Complementar

- Aula 44 - POO: Herança Múltipla
 - <https://youtu.be/XB3n7ihzis4>
- Aula 45 - POO: Herança Múltipla e Superposição de Métodos
 - https://youtu.be/GI19t0p_5y0
- Aula 46 - POO: Herança Virtual
 - https://youtu.be/t5Iz1_reJu0